IOT MINI PROJECT · PHASE 2: FINAL REPORT

Addressing Class Imbalance in CICIoT2023-Based Intrusion Detection via SMOTE and Feature Selection

An IoT gateway intrusion detection system designed with the 10-step IoT design methodology

Dhruv Sandilya · 231CS122
Department of Computer Science and Engineering
National Institute of Technology Karnataka, Surathkal

Base paper: E. C. P. Neto et al., "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors* 23(13):5941, 2023.
November 2026

Abstract

The CICIoT2023 dataset of Neto et al. contains 46,686,579 labelled network flows from a 105-device IoT testbed under 33 attacks in seven categories. Its benchmark reports high accuracy, but the data are extremely imbalanced (DDoS and DoS make up 90.1% of the flows; there are 2,588 DDoS flows for every brute-force flow), and rare attacks are largely missed. This project designs an IoT gateway intrusion detection system following the 10-step IoT design methodology and, as its application, (1) reproduces the paper's Random Forest benchmark on a de-duplicated 3.0% sample (1,401,354 flows from all 169 files), (2) improves detection of the rare classes by handling class imbalance on the training data only, comparing SMOTE, Borderline-SMOTE, SMOTE with undersampling, class weights and decision-threshold tuning across four models, and (3) extends the work with feature selection for resource-constrained gateways. The reproduced Random Forest matches the paper's accuracy (0.995 vs 0.994 for 8 classes) and confirms its blind spot: web-based and brute-force attacks are recalled only 16% and 24% of the time. For the Random Forest, every imbalance method improves macro-F1 significantly (paired bootstrap, $p < 0.05$). Threshold tuning gives the best Random Forest (macro-F1 0.785 $\rightarrow$ 0.851) at no training or size cost, and SMOTE raises rare-class recall from 0.398 to 0.626. With only 20 of the 46 features the tuned Random Forest keeps its macro-F1 (0.856) while predicting 22% faster with a 28% smaller model, and a class-weighted Decision Tree reaches macro-F1 0.841 in 1.1 MB, about 155 times smaller than the Random Forest. We also find that 15.9% of test rows have an exact copy in the training data unless duplicates are removed, and that the paper's precision and recall columns are swapped.

Contents

1. Introduction
2. Background and related work
3. IoT system design: the 10-step methodology
    a. Purpose and requirements
    b. Process model
    c. Domain model
    d. Information model
    e. Service specifications
    f. IoT level
    g. Functional view
    h. Operational view
    i. Device and component integration
    j. Application development
4. Experimental methodology
5. Results
    a. Data and imbalance
    b. Reproduction
    c. Improvement
    d. Extension: feature selection
    e. Effect of duplicate rows
6. Discussion
7. Conclusion and future work

References · Appendix A: Reproducibility

1. Introduction

1.1 Motivation

IoT deployments add many small, rarely patched devices to home and industrial networks, and each one widens the attack surface. Network intrusion detection systems (IDS) that learn from labelled traffic are a natural defence, and the CICIoT2023 dataset [1] was built to train and benchmark them on realistic IoT traffic. Its class distribution, however, is extremely skewed: a detector that always answers “DDoS” is already 73% accurate while never catching a brute-force login or a web injection, the attacks that lead to device takeover rather than mere unavailability. Headline accuracy therefore says little about the attacks that matter most; per-class recall does.

1.2 Base paper

Neto et al. [1] captured traffic in a 105-device smart-home testbed (67 IP devices plus 38 Zigbee/Z-Wave devices behind five hubs), executed 33 attacks from malicious IoT devices, and published 46,686,579 labelled rows with 46 features computed over windows of 10 or 100 packets. They benchmarked Logistic Regression, Perceptron, AdaBoost, Random Forest and a deep neural network on 2-, 8- and 34-class tasks with an 80/20 split and standard scaling, with no treatment of class imbalance. Their confusion matrix (Table 8 of [1]) shows the Random Forest recalling only 4.2% of web-based and 5.7% of brute-force flows. Their conclusion names “the analysis of features and how they influence different ML models” as future work.

1.3 Objectives and contributions

The project answers three research questions, each with a measurable target set in Phase 1:

- **RQ1 – Reproduction:** does a Random Forest (plus Decision Tree, LightGBM and Balanced Random Forest) reproduce the paper's benchmark? *Answer: yes; accuracy matches to within 0.002 at all three granularities (Section 5.2).*
- **RQ2 – Improvement:** does handling the imbalance raise recall on the rare classes, significantly? *Answer: yes; for the Random Forest every method beats the baseline ($p < 0.05$), with rare-class recall up by as much as 0.228 and macro-F1 up by 0.066 (Section 5.3).*
- **RQ3 – Extension:** can a small feature subset keep that gain at lower cost? *Answer: yes; 20 of 46 features keep the full macro-F1 with faster prediction and a smaller model (Section 5.4).*

Additional findings: duplicate rows leak 15.9% of the test set into training unless removed; the precision and recall columns of the paper's Table 6 are swapped; and LightGBM with default settings is unstable on this imbalance unless the training data are rebalanced.

1.4 Organisation

Section 2 covers background. Section 3 designs the IoT system with the 10-step IoT design methodology of Bahga and Madisetti [6]; step 10, application development, is the machine-learning pipeline, whose methodology and results are given in Sections 4 and 5. Section 6 discusses the results and Section 7 concludes.

2. Background and related work

Datasets. CICIoT2023 [1] is distinguished by its scale and by using IoT devices as attackers. The course reading list adds CIC IoT-DIAD 2024 (device identification and anomaly detection) [15] and the Dalhousie NIMS Lab IoT 2024 dataset [16], both candidates for testing how well models trained here transfer.

IoT intrusion detection. Deep-learning IDS have been proposed for MQTT-based IoT [13], and LightGBM combined with metaheuristic feature selection has been used for MQTT-IoT intrusion detection [14]. Tree ensembles such as Random Forest [7] remain strong, cheap baselines on tabular flow features, which is why this project builds on them.

Class imbalance. Data-level methods change the training distribution: SMOTE [2] synthesises minority samples by interpolating between neighbours, Borderline-SMOTE [3] does so only near the class boundary, and random undersampling removes majority samples. Algorithm-level methods leave the data unchanged: class weights scale each class's loss, threshold moving changes the decision rule after training, and the Balanced Random Forest [4] trains every tree on a class-balanced bootstrap sample. Gradient boosting with LightGBM [5] is included as a strong modern learner.

Feature selection. Mutual information between each feature and the label [17], impurity importance of a Random Forest [7], and recursive feature elimination [18] are standard filter, embedded and wrapper methods.

IoT design methodology. Bahga and Madisetti [6] describe ten steps from purpose and requirements to application development, with six deployment levels; Section 3 applies them to this system.

3. IoT system design: the 10-step methodology

The system is an intrusion detection system for an IoT network, deployed at the network gateway. Figure 1 summarises the ten design steps of Bahga and Madisetti [6] as applied here.

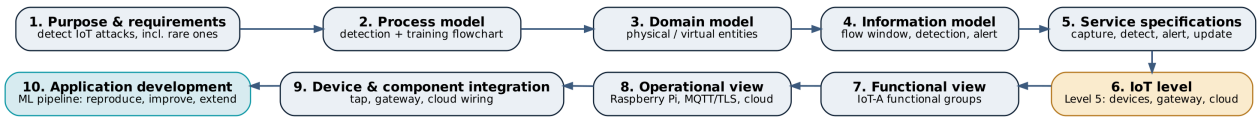


Figure 1. The ten steps of the IoT design methodology applied to the gateway intrusion detection system.

3.1 Step 1: Purpose and requirements specification

Table 1. Purpose and requirements.

Aspect	Specification
Purpose	Detect and classify malicious traffic in an IoT network at its gateway, including rare attack types, so an administrator can respond.
Behaviour	Observe the traffic of every connected device, summarise it into packet windows, classify each window as benign or one of 7 attack categories (33 attack types), and raise alerts.
System management	Monitor the gateway's health and throughput remotely; update the detection model remotely.
Data analysis	Real-time inference at the gateway (edge); training, rebalancing and feature selection in the cloud.
Application deployment	Detection on the gateway; storage, dashboard and model registry in the cloud.
Security	Encrypted, authenticated alert and update channels (TLS); integrity check of every model before installation; the IDS observes a copy of the traffic so it never blocks the network if it fails.
Measurable targets	Rare-class recall and macro-F1 above the reproduced baseline (significant at $p < 0.05$); prediction latency in microseconds per flow; model size small enough for a Raspberry Pi-class gateway.

3.2 Step 2: Process model specification

The process model (Figure 2) describes *how* the system works as a flowchart with two loops: an online loop on the gateway that classifies every traffic window and raises alerts, and an offline loop in the cloud that trains, evaluates and publishes models. The two meet through the model update service and the stored alerts.

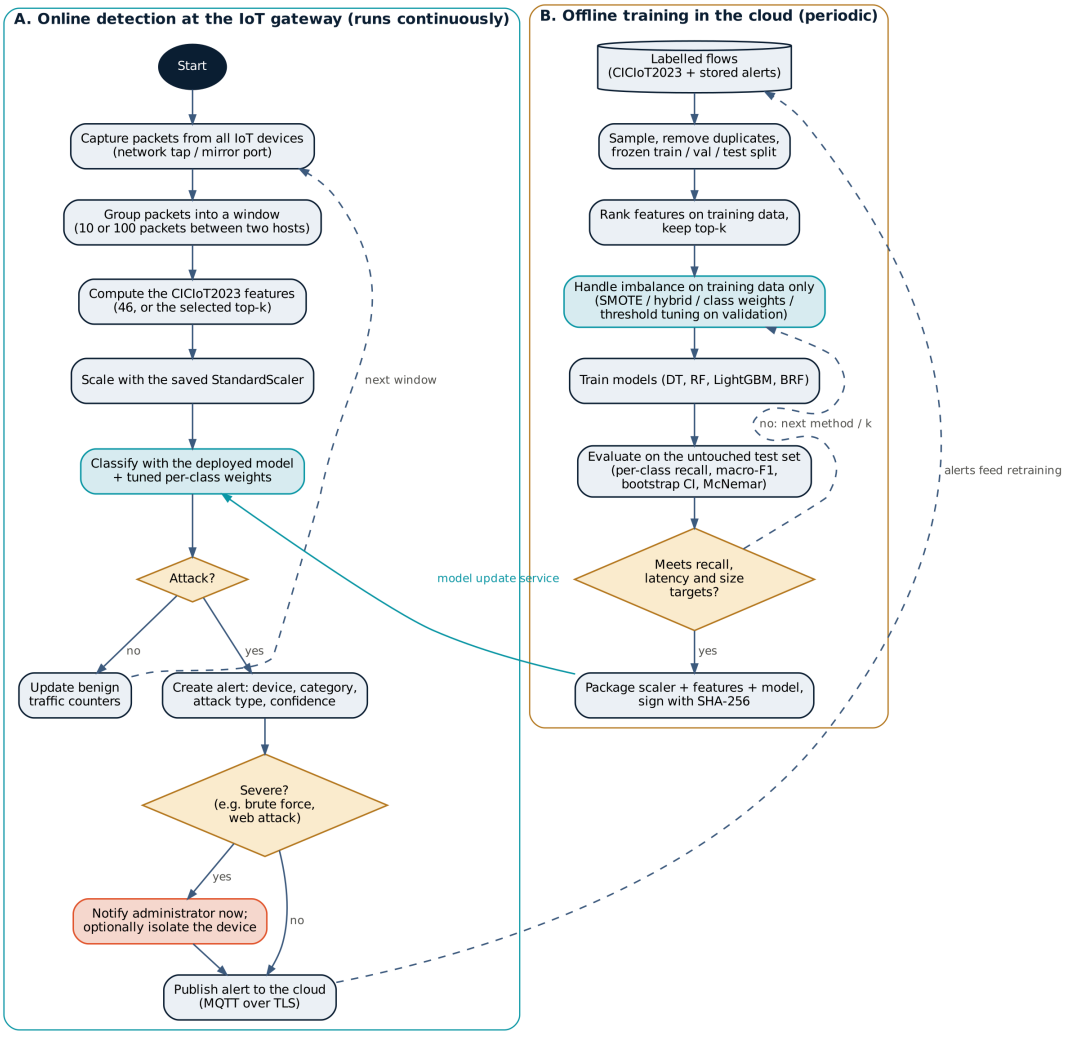


Figure 2. Process model: online detection at the gateway (A) and offline training in the cloud (B).

3.3 Step 3: Domain model specification

The domain model (Figure 3) names the entities, objects and relationships independently of any technology. **Physical entities** are the IoT devices, the malicious devices and the traffic they exchange. Each is represented by **virtual entities**: a device profile, the flow windows of its traffic, detection results, alerts, and the model itself. **Devices** (the gateway with its network tap, and a cloud server) monitor the physical entities and host **resources**: on-device resources (capture, feature extractor, classifier, local log) and network resources (alert store, model registry, training compute). **Services** expose those resources to the administrator.

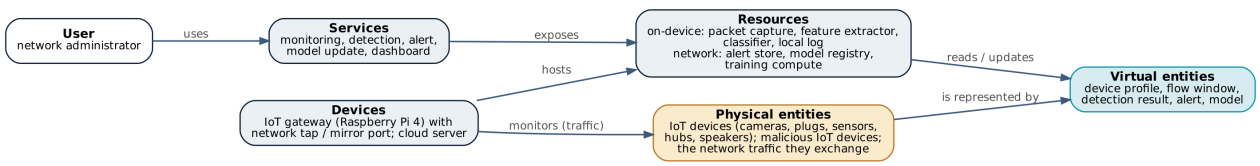


Figure 3. Domain model: physical and virtual entities and the devices, resources and services that link them.

3.4 Step 4: Information model specification

The information model (Figure 4) defines the data structures of the virtual entities and how they relate. The central structure is the **FlowWindow**, which carries the 46 CICIoT2023 features of one window of 10 or 100 packets between two hosts [1]; a Detection refers to one window and to the ModelArtifact that produced it, and an Alert groups detections for one device.

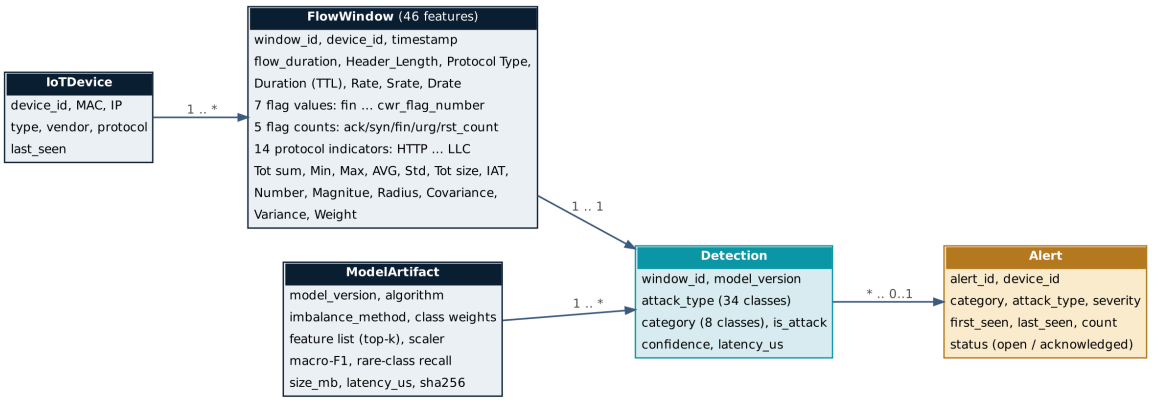


Figure 4. Information model: virtual entities, their attributes and their relationships.

3.5 Step 5: Service specifications

Table 2. Service specifications.

Service	Type	Input	Output	Schedule	Hosted on
Traffic capture	Native (libpcap/tcpdump)	Mirrored packets	Packet stream	Continuous	Gateway
Feature extraction	Native (Python)	Packets	FlowWindow (46 features)	Per window	Gateway
Detection	Native (scikit-learn / LightGBM)	FlowWindow (top-k, scaled)	Detection	Per window, µs	Gateway
Alert	MQTT publish over TLS	Detection with is_attack = true	Alert (JSON)	On event	Gateway → cloud
Storage	Database API	Alerts, statistics	Stored records	On event	Cloud
Model training	Batch job (Kaggle / cloud VM)	Labelled flows	ModelArtifact + metrics	Periodic	Cloud
Model update	REST over HTTPS (pull)	ModelArtifact + SHA-256	Installed model	Daily poll	Cloud → gateway
Dashboard	Web (HTTPS)	Alerts, statistics	Charts, notifications	On demand	Cloud

3.6 Step 6: IoT level specification

The system is a **Level 5** deployment (Figure 5): many end nodes (the IoT devices, which only generate traffic), one coordinator node (the gateway, which collects all traffic) and the cloud for storage, analysis and the application. It departs from the textbook Level 5 in one deliberate way: detection runs *at the coordinator* rather than in the cloud, so that attacks are caught within microseconds without shipping all traffic upstream. The feature-selection extension (Section 5.4) exists to make that edge inference cheap. Table 3 explains why the other levels do not fit.

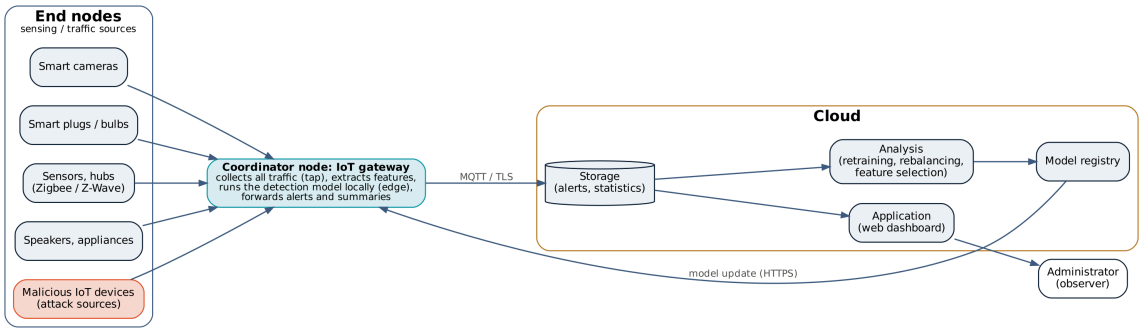


Figure 5. Level 5 deployment: IoT devices as end nodes, the gateway as coordinator with edge detection, and the cloud.

Table 3. Choosing the IoT level.

Level	Definition [6]	Fit for this system
1	Single node; local storage and analysis	No: many devices must be monitored
2	Single node; cloud storage, local analysis	No: single node
3	Single node; cloud storage and analysis	No: single node; raw traffic to the cloud is too costly
4	Multiple nodes, each analysing locally; cloud storage	No: IoT devices are too constrained to run an IDS each
5	**Multiple end nodes and one coordinator; cloud storage and application**	**Yes: devices are end nodes, the gateway is the coordinator**
6	Independent end nodes sending to the cloud; central controller	No: devices cannot run agents; all traffic to the cloud adds latency

3.7 Step 7: Functional view specification

The functional view (Figure 6) maps the entities and services onto the IoT-A functional groups (FG): Device, Communication, Services, Management, Security and Application.

Application FG	Web dashboard: alerts by device, category and time		Administrator notifications
Management FG	Model registry and versioning	Gateway configuration (top-k features, class weights)	Gateway health monitoring
Services FG	Capture + feature extraction	Detection (classifier + tuned weights)	Alert; model update; training
Communication FG	Links: Wi-Fi, Ethernet, Zigbee, Z-Wave	Network / transport: IPv4, TCP, UDP	Application: MQTT (alerts), HTTPS (updates, dashboard)
Device FG	IoT devices (105 in the CICIoT2023 testbed)	IoT gateway: Raspberry Pi 4 + network tap	Cloud server
Security FG (spans all layers)	TLS for MQTT and HTTPS, client certificates	SHA-256 check before a model is installed	Dashboard authentication and access control

Figure 6. Functional view: components of each functional group.

3.8 Step 8: Operational view specification

Table 4. Operational view: how each functional group is deployed and operated.

Functional group	Operational choice
Device	Raspberry Pi 4 gateway with a network tap or switch mirror port; cloud VM
Communication	MQTT broker (e.g. Mosquitto) with TLS for alerts; HTTPS for model updates and the dashboard
Services hosting	Capture, extraction and detection run natively on the gateway; storage, dashboard and registry on the cloud VM
Storage	Relational / time-series database for alerts; object storage for model artifacts
Training	Batch training on Kaggle CPU (used in this project; 2.7 h for the full experiment matrix)
Application hosting	Web dashboard on the cloud VM
Management	Versioned model artifacts with metrics and SHA-256; gateway configuration file (top-k list, class weights)
Security	X.509 client certificates for MQTT; TLS everywhere; checksum check before a model is installed

3.9 Step 9: Device and component integration

Figure 7 shows how the components are wired. Devices connect to an access point or switch as usual; a network tap (as in the CICIoT2023 testbed [1]) or a switch mirror port gives the gateway a passive copy of all traffic, so a failure of the IDS never interrupts the network. The gateway publishes alerts to the cloud and pulls model updates.

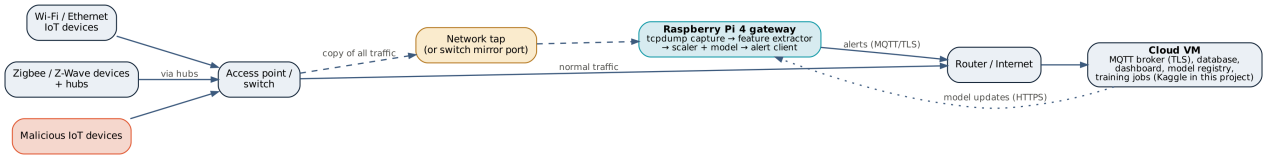


Figure 7. Device and component integration.

3.10 Step 10: Application development

The application's core is the detection model and the pipeline that produces and evaluates it. It was built as a tested Python package (scikit-learn [8], imbalanced-learn [9], LightGBM [5]) and run end to end on Kaggle; Table 5 lists its modules. Sections 4 and 5 describe the experiments that choose the model for the gateway: reproduction, improvement and extension.

Table 5. Software modules of the application (repository: github.com/Dhruvsandilya0711/IoT_Mini_Project).

Module	Role
src/load.py	Streams the 169 CSV files and keeps a random 3% of rows (class ratios preserved)
src/preprocess.py	Removes NaN/inf rows and exact duplicates; frozen stratified train/val/test split
src/resample.py, src/threshold.py	Imbalance methods (training data only); per-class threshold tuning on validation
src/train.py	Trains and evaluates one configuration; logs metrics, latency, size, predictions
src/feature_selection.py	Mutual information, RF importance and RFE rankings on training data
src/stats.py, src/analyze.py	Bootstrap confidence intervals, McNemar tests, tables and figures
scripts/kaggle_run.py	Runs every phase on Kaggle and packs the results

4. Experimental methodology

4.1 Data

All 169 CSV files of the original release were streamed and each row kept with probability 0.0300, giving 1,401,354 rows (3.00% of 46,686,579) with the true class ratios. No row had missing or infinite values. 166,898 exact duplicate rows (11.9%) were removed, and the remaining 1,234,456 rows were split once, stratified on all 34 classes, into training (888,807), validation (98,757) and test (246,892) sets. The test set is never resampled and never used for any choice. Classes below 1% of the training data (Recon, Web and BruteForce) are the *rare classes*; their mean recall is reported as rare-class recall.

4.2 Models

Decision Tree; Random Forest (100 trees) as in [1]; LightGBM (100 boosting rounds, 63 leaves, learning rate 0.1); and Balanced Random Forest (100 trees, each on a class-balanced bootstrap sample). All other settings are library defaults; features are standardised as in [1].

4.3 Imbalance methods (training data only)

- **SMOTE:** every class with fewer than 50,000 training rows is raised to 50,000 with synthetic samples (k = 5); an ablation tries 10,000 and 100,000.
- **SMOTE + undersampling (hybrid):** classes above 200,000 rows are randomly cut to 200,000, then SMOTE as above.
- **Borderline-SMOTE:** SMOTE restricted to minority samples near the class boundary.
- **Class weights:** each class's loss weighted inversely to its frequency.
- **Threshold tuning:** the model is trained on the original data; one multiplicative weight per class is then applied to its predicted probabilities, chosen by coordinate ascent over 22 values between 0.1 and 100 to maximise macro-F1 on the *validation* set.

4.4 Feature selection

Features are ranked on the training data only, by mutual information [17] and Random Forest importance (200,000-row stratified subsample) and by RFE [18] with a 50-tree forest (50,000 rows). Each model is then retrained on the top $k \in \{5, 10, 15, 20, 30\}$ features of the mutual-information ranking with the imbalance method that gave its best *validation* macro-F1.

4.5 Metrics and statistics

Accuracy, macro-averaged precision, recall and F1, balanced accuracy, Matthews correlation, per-class recall and rare-class recall on the test set; prediction latency (batch prediction time per flow on a 4-core Kaggle CPU) and model size (serialised bytes). Each 8-class configuration is run with three seeds (mean ± standard deviation). Each method is compared with the baseline trained with the same seed on the same test rows by a paired bootstrap (1,000 resamples; 95% confidence interval and one-sided p-value) [11] and McNemar's exact test on per-row correctness [10].

4.6 Execution

The full matrix (123 training runs) ran unattended on a free Kaggle CPU session in 2.7 hours.

5. Results

5.1 Data and imbalance

Table 6 and Figure 8 confirm the imbalance reported in [1]: the sample's category shares match the paper's Table 3 to within sampling error, and the rarest category (BruteForce) is 2,588 times smaller than DDoS. Only 153 Web and 79 BruteForce flows reach the test set, which is why their recall carries the widest confidence intervals. Two features, Telnet and SMTP, are always zero in the sample.

Table 6. Class distribution of the sample (before de-duplication for counts and shares; after it for the split).

Category	Rows in sample	Share	DDoS : class	Train rows	Test rows
DDoS	1,019,793	72.772%	1 : 1	622,617	172,951
DoS	243,434	17.371%	4 : 1	168,846	46,901
Mirai	78,728	5.618%	13 : 1	54,576	15,161
Benign	33,337	2.379%	31 : 1	24,003	6,667
Spoofing	14,522	1.036%	70 : 1	10,456	2,904
Recon	10,382	0.741%	98 : 1	7,476	2,076
Web	764	0.055%	1,335 : 1	550	153
BruteForce	394	0.028%	2,588 : 1	283	79

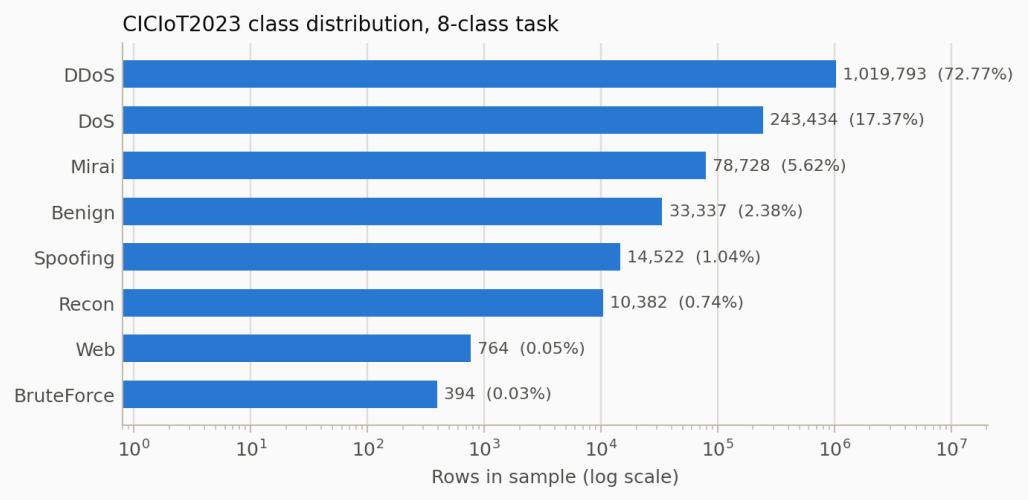


Figure 8. Class distribution of the 8-class task (log scale).

5.2 Reproduction (RQ1)

Table 7 compares the reproduced Random Forest with the paper. Accuracy agrees to within 0.002 at every granularity. Comparing precision and recall needs care: the 34-class Logistic Regression row of the paper's Table 6 (accuracy 0.80231507, “recall” 0.59520185, “precision” 0.486752461, F1 0.49388408) matches, digit for digit, the output of the public CICIoT2023 example notebook, which calls `recall_score(y_pred, y_test)` and `precision_score(y_pred, y_test)` with the arguments in the wrong order; scikit-learn expects `(y_true, y_pred)`, so the two metrics come out swapped. Recomputing the metrics from the paper's own Random Forest confusion matrix (its Table 8) confirms this: macro recall is 0.706 and macro precision 0.917, the reverse of the Table 6 labels (0.910 and 0.705), while accuracy (0.994) and F1 (0.720) match. We therefore compare the paper's “recall” with our precision and vice versa.

Table 7. Random Forest: base paper (Table 6 of [1]) vs this reproduction. Ours are means over seeds for 8 classes and one seed otherwise.

Task	Accuracy, paper	Accuracy, ours	Macro-F1, paper	Macro-F1, ours	Paper: printed recall / precision	Ours: precision / recall
2 classes	0.997	0.997	0.965	0.970	0.965 / 0.965	0.968 / 0.972
8 classes	0.994	0.995	0.719	0.785	0.910 / 0.705	0.945 / 0.750
34 classes	0.992	0.993	0.714	0.751	0.832 / 0.704	0.815 / 0.734

Our macro-F1 is higher than the paper's for 8 and 34 classes. Possible reasons are the different split (random, stratified, de-duplicated), library versions and training procedure: the same example notebook calls `fit()` once per CSV file, which in scikit-learn retrains the model from scratch each time, so a model trained that way would see only the last file (about 0.6% of the training data). Crucially, the qualitative picture is reproduced (Figure 9): DDoS, DoS and Mirai are recalled almost perfectly, while Web and BruteForce are mostly mistaken for Benign, Recon and Spoofing, exactly as in the paper's Table 8 (Figure 10).

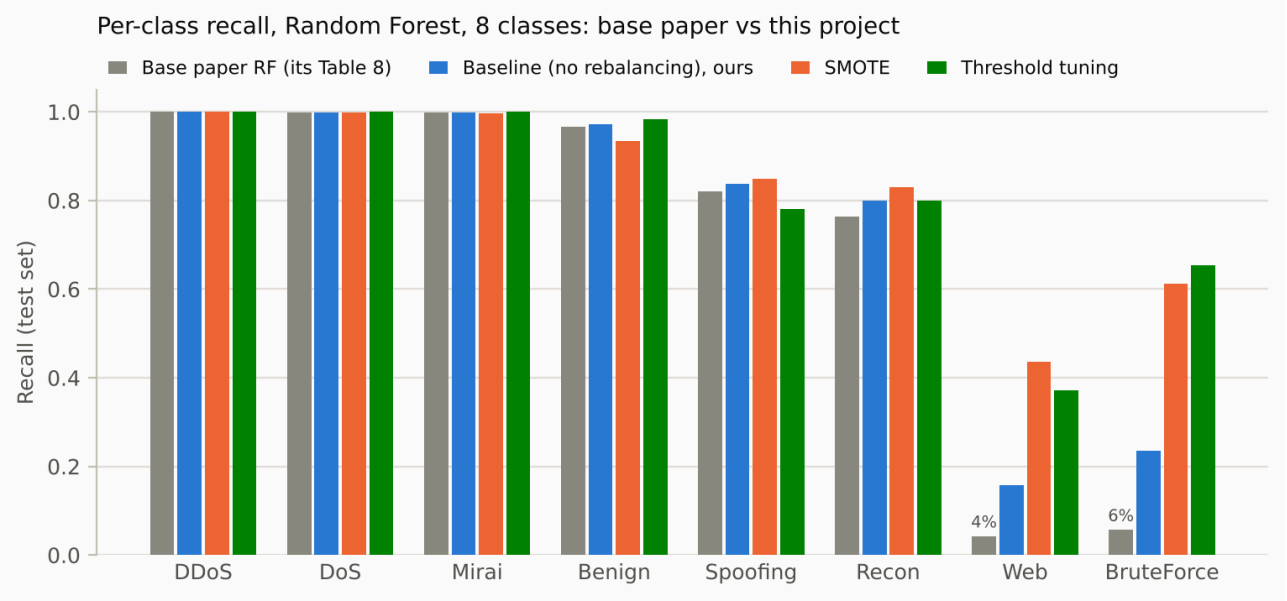


Figure 9. Per-class recall of the Random Forest, 8 classes: the base paper (from its Table 8) and this project's baseline, SMOTE and threshold-tuned models (mean over three seeds).

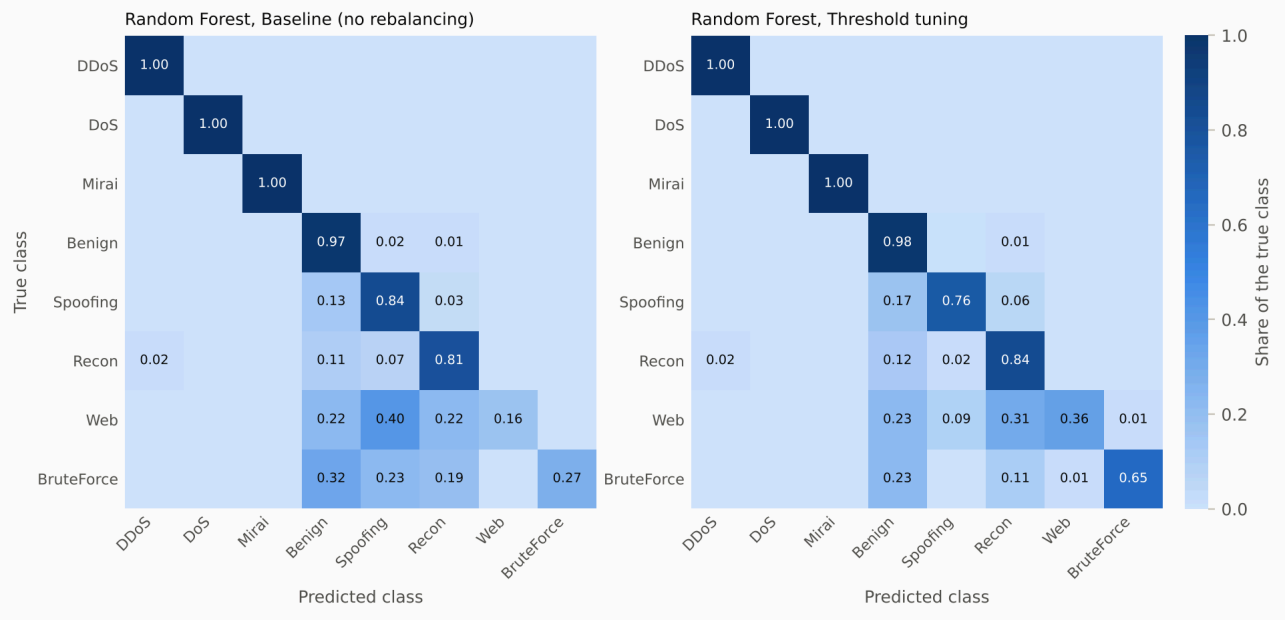


Figure 10. Random Forest confusion matrices (8 classes, seed 0), row-normalised: baseline vs threshold tuning.

5.3 Improvement (RQ2)

Random Forest, 8 classes. Table 8 compares the six training regimes. Every imbalance method raises macro-F1 and rare-class recall over the baseline, and Table 9 shows that every gain is statistically significant: the 95% bootstrap confidence intervals of the change in macro-F1 exclude zero for every method and every seed.

Table 8. Random Forest, 8 classes, test set (mean over three seeds; ± is the standard deviation).

Method	Macro-F1	Balanced acc.	Rare-class recall	Web recall	BruteForce recall	Accuracy	Size (MB)	Train (s)
Baseline (no rebalancing)	0.785 ± 0.006	0.750	0.398	0.157	0.236	0.995	169	61
SMOTE	0.837 ± 0.002	0.832	0.626	0.436	0.612	0.994	468	103
SMOTE + undersampling	0.837 ± 0.001	0.833	0.626	0.429	0.612	0.994	449	73
Class weights	0.841 ± 0.007	0.795	0.523	0.314	0.477	0.994	227	62
Borderline-SMOTE	0.811 ± 0.002	0.779	0.481	0.355	0.262	0.994	487	112
Threshold tuning	0.851 ± 0.002	0.824	0.609	0.373	0.654	0.995	169	65

Table 9. Each method vs the Random Forest baseline trained with the same seed, on the same test rows.

Method	Mean Δ macro-F1	95% CI (all seeds)	Max bootstrap p	Mean Δ rare-class recall	McNemar p < 0.05
SMOTE	+0.052	[+0.027, +0.077]	< 0.001	+0.228	3 / 3
SMOTE + undersampling	+0.051	[+0.028, +0.081]	< 0.001	+0.228	3 / 3
Class weights	+0.056	[+0.025, +0.081]	< 0.001	+0.125	3 / 3
Borderline-SMOTE	+0.026	[+0.002, +0.053]	0.013	+0.084	3 / 3
Threshold tuning	+0.066	[+0.042, +0.090]	< 0.001	+0.211	2 / 3

Threshold tuning gives the best Random Forest (macro-F1 0.851, +0.066) with no extra training data, the same model size as the baseline (169 MB) and four seconds of tuning. It lifts BruteForce recall from 0.236 to 0.654 and Web recall from 0.157 to 0.373. **SMOTE** and the hybrid give the largest rare-class recall (0.626) but at a cost in precision (Web precision falls from 0.901 to 0.443), and the forest trained on synthetic data is 2.8 times larger. The SMOTE ablation shows the same trade-off: raising rare classes to 10,000 rows gives higher rare-class recall (0.668) than 50,000 or 100,000 rows. **Borderline-SMOTE** helps least, because few rare-class samples lie near a boundary in this feature space. For threshold tuning, McNemar's test is not significant for one seed although the macro-F1 gain is: it trades a few majority-class errors for many rare-class detections, which leaves the overall count of correct rows almost unchanged while the per-class balance improves.

All models. Table 10 ranks every model and method. The class-weighted Decision Tree (macro-F1 0.841) is almost as good as the best Random Forest at 1.1 MB and 0.11 µs per flow, and even the untuned Decision Tree beats the untuned Random Forest on macro-F1, most likely because a forest's averaged vote dilutes the few trees that recognise a rare class. For the Decision Tree, oversampling raises rare-class recall (0.656 → 0.704 with SMOTE) but lowers macro-F1, because a single tree fits the synthetic points closely and loses precision; class weights improve both slightly. Threshold tuning cannot change a fully grown Decision Tree, whose leaves are pure and give 0/1 probabilities. The Balanced Random Forest has the highest rare-class recall (0.771) but the lowest precision, so its macro-F1 is only 0.705. **LightGBM** with default settings is unstable on this imbalance: without rebalancing its macro-F1 varies from seed to seed (0.503 ± 0.126), which

points to numerical instability of the boosted multi-class model when rare classes contribute tiny gradients; rebalancing removes the problem (hybrid: 0.833 ± 0.002). Its large gains in Table 10 therefore reflect a repaired baseline as much as an improvement.

Table 10. All models and methods, 8 classes, all 46 features, ranked by macro-F1 (mean ± std over three seeds).

Model	Method	Macro-F1	Rare-class recall	Worst-class recall	Latency (µs/flow)	Size (MB)
Random Forest	Threshold tuning	0.851 ± 0.002	0.609	0.373	4.02	168.8
Random Forest	Class weights	0.841 ± 0.007	0.523	0.314	4.23	227.0
Decision Tree	Class weights	0.841 ± 0.000	0.680	0.549	0.11	1.1
Random Forest	SMOTE	0.837 ± 0.002	0.626	0.436	4.00	468.4
Random Forest	SMOTE + undersampling	0.837 ± 0.001	0.626	0.429	3.92	449.2
LightGBM	SMOTE + undersampling	0.833 ± 0.002	0.700	0.549	21.66	5.6
LightGBM	SMOTE	0.832 ± 0.002	0.690	0.551	21.25	5.6
Decision Tree	Baseline (no rebalancing)	0.830 ± 0.001	0.656	0.516	0.11	0.8
Decision Tree	Threshold tuning	0.830 ± 0.001	0.656	0.516	0.12	0.8
Decision Tree	Borderline-SMOTE	0.820 ± 0.002	0.647	0.544	0.12	2.8
Decision Tree	SMOTE + undersampling	0.812 ± 0.001	0.706	0.608	0.12	2.7
Decision Tree	SMOTE	0.812 ± 0.003	0.704	0.599	0.12	2.7
Random Forest	Borderline-SMOTE	0.811 ± 0.002	0.481	0.262	4.26	486.6
LightGBM	Class weights	0.802 ± 0.006	0.582	0.344	21.53	5.6
Random Forest	Baseline (no rebalancing)	0.785 ± 0.006	0.398	0.157	3.91	168.8
Balanced Random Forest	Baseline (no rebalancing)	0.705 ± 0.002	0.771	0.695	2.97	29.7
LightGBM	Borderline-SMOTE	0.558 ± 0.137	0.526	0.313	22.44	5.6
LightGBM	Baseline (no rebalancing)	0.503 ± 0.126	0.181	0.046	26.19	5.5
LightGBM	Threshold tuning	0.358 ± 0.141	0.088	0.021	25.11	5.5

34 classes. The 34-class task is harder still (Table 11). The baseline Random Forest misses 4 attack types completely (Backdoor_Malware, Recon-PingSweep, SqlInjection, Uploading_Attack). SMOTE gives the largest gain (macro-F1 0.751 → 0.807). These rarest types have only 8 to 33 test rows each, so their individual recalls are noisy.

Table 11. Random Forest, 34 classes (one seed).

Method	Macro-F1	Δ vs baseline	Rare-class recall	Size (MB)
Baseline (no rebalancing)	0.751	–	0.526	777
SMOTE	0.807	+0.056 ($p < 0.001$)	0.650	2887
Class weights	0.788	+0.037 ($p < 0.001$)	0.566	1019
Threshold tuning	0.789	+0.038 ($p < 0.001$)	0.595	777

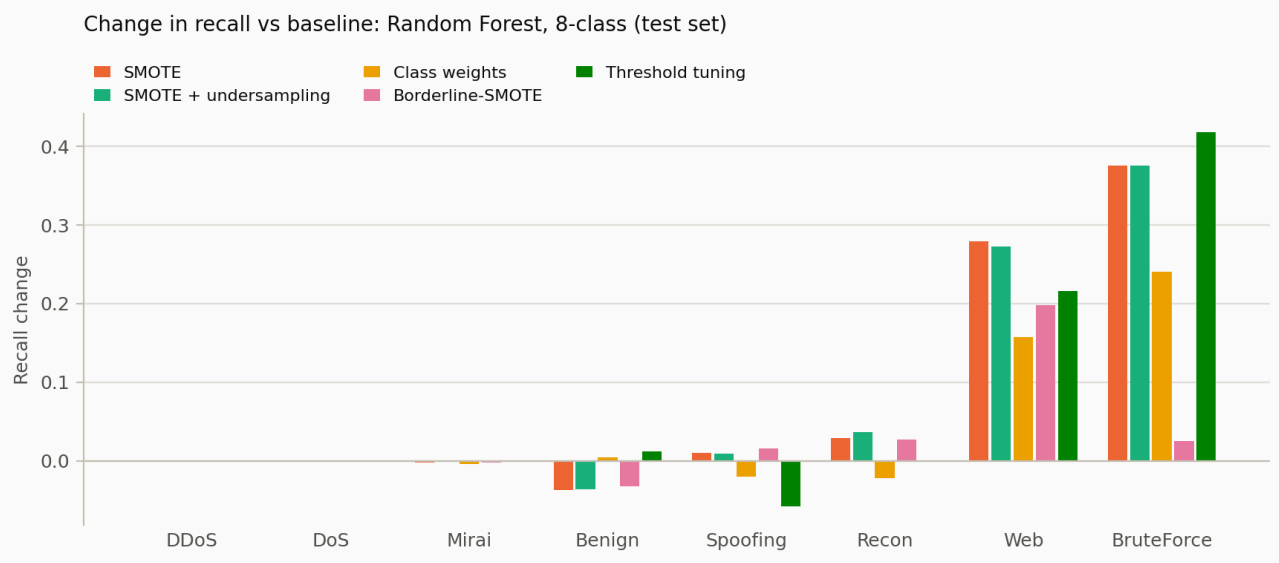


Figure 11. Change in per-class recall over the baseline, Random Forest, 8 classes.

5.4 Extension: feature selection (RQ3)

The three rankings largely agree (Table 12): 8 features are in all three top-10 lists (Header_Length, IAT, Magnitue, Max, Min, Protocol Type, Tot size, Tot sum). The inter-arrival time (IAT) ranks first in every method; packet-size statistics and flow duration follow.

Table 12. Top-10 features by each ranking (training data only).

Rank	Mutual information	RF importance	RFE
1	IAT	IAT	IAT
2	Tot size	Min	Min
3	Max	flow_duration	Magnitue
4	Magnitue	Protocol Type	flow_duration
5	AVG	Tot size	Tot size
6	Tot sum	Magnitue	Max
7	Min	Max	Tot sum
8	Header_Length	AVG	rst_count
9	Protocol Type	Header_Length	Protocol Type
10	Std	Tot sum	Header_Length

Table 13 and Figure 12 show detection quality and cost as features are removed (mutual-information ranking). The threshold-tuned Random Forest loses nothing down to 20 features: macro-F1 is 0.856 with 20 features against 0.851 with all 46, prediction takes 3.12 instead of 4.02 µs per flow (22% faster), and the model shrinks from 169 to 121 MB (28% smaller). With 10 features macro-F1 is still 0.841, well above the full-feature baseline (0.785), at 2.58 µs per flow. The Phase 1 target (at most 15 features keeping most of the gain) is met: with 15 features the tuned forest keeps 92% of its macro-F1 gain. Rare-class recall falls faster than macro-F1 as features are removed, so 20 features is the better operating point for a detector whose job is rare attacks.

Table 13. Feature sweep, 8 classes: threshold-tuned Random Forest and class-weighted Decision Tree (mean over three seeds).

k	RF macro-F1	RF rare recall	RF µs/flow	RF MB	DT macro-F1	DT µs/flow	DT MB
5	0.837	0.551	2.16	146	0.795	0.073	1.85
10	0.841	0.554	2.58	134	0.795	0.081	1.63
15	0.846	0.572	3.11	139	0.812	0.087	1.28
20	0.856	0.599	3.12	121	0.828	0.089	1.13
30	0.856	0.613	3.56	134	0.828	0.104	1.11
46	0.851	0.609	4.02	169	0.841	0.112	1.09

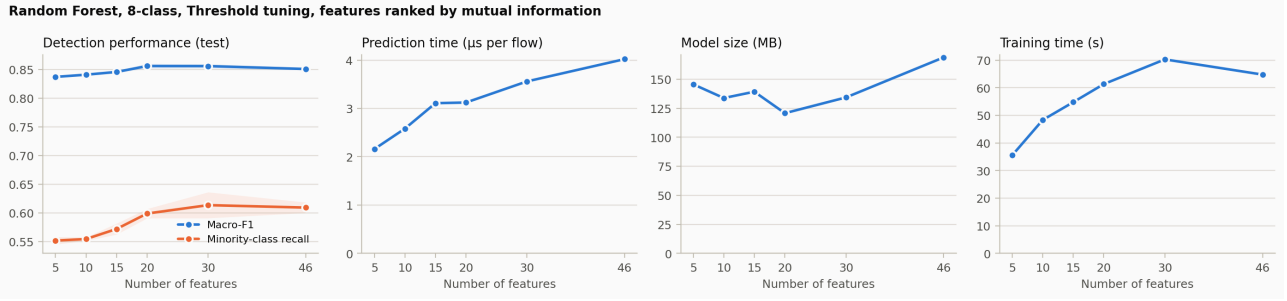


Figure 12. Threshold-tuned Random Forest against the number of features: detection quality, prediction time, model size and training time.

For a Raspberry Pi-class gateway the cost axis matters as much as accuracy (Figure 13). The class-weighted Decision Tree is the most deployable model: about 1 MB and 0.1 µs per flow with macro-F1 0.841, against 169 MB and 4.0 µs for the best forest.

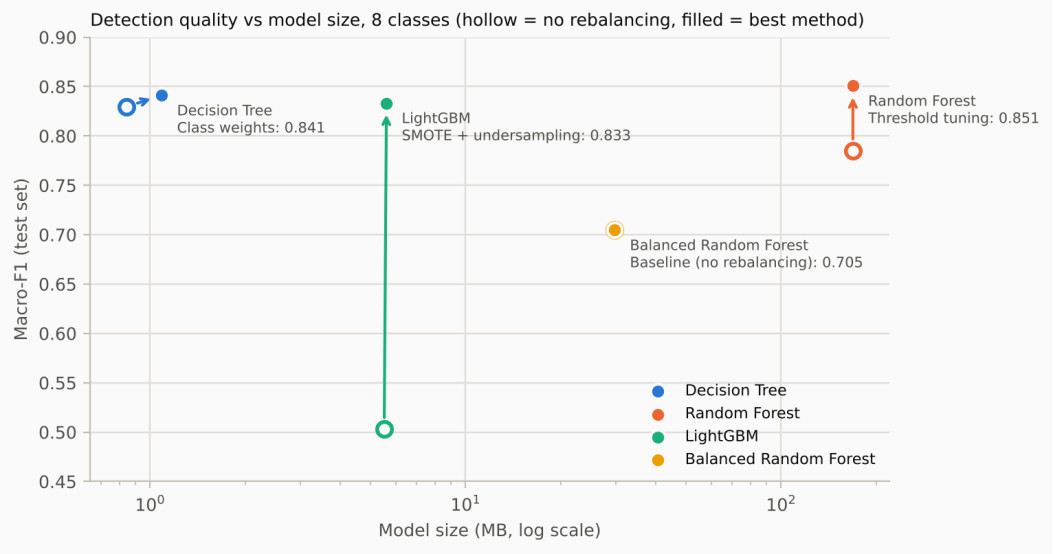


Figure 13. Macro-F1 against model size for each model: baseline (hollow) and best imbalance method (filled).

5.5 Effect of duplicate rows

Without de-duplication, 44,441 test rows (15.9%) have an exact copy in the training data. Table 14 shows the effect on the baseline: accuracy is slightly higher with duplicates, because the duplicated rows are mostly easy DDoS and DoS flows, while macro-F1 is not, because the copies do not help the rare classes. Removing duplicates is therefore the honest default, and it does not explain away the rare-class problem.

Table 14. Baselines with and without exact duplicate rows (8 classes).

Data	Model	Accuracy	Macro-F1	Rare-class recall
with duplicates	Random Forest	0.995	0.768	0.359
with duplicates	Decision Tree	0.994	0.817	0.610
duplicates removed	Decision Tree	0.994	0.830	0.656
duplicates removed	Random Forest	0.995	0.785	0.398

6. Discussion

What fixes the blind spot. The baseline's errors are not random: rare attacks are pulled towards the classes that surround them in feature space (Benign, Recon, Spoofing). Every remedy moves the decision boundary towards the rare classes. Threshold tuning does it most cheaply, after training, by re-weighting the forest's votes; it is the recommended default because it costs no extra data, time or memory and gives the best macro-F1. SMOTE does it by adding synthetic rare samples, which recovers more rare attacks but raises false alarms and makes the forest much larger.

Precision vs recall. For an IDS, a missed brute-force login is usually worse than a false alarm, so the higher rare-class recall of SMOTE or the Balanced Random Forest may be preferred where an analyst triages alerts; where alert fatigue matters, threshold tuning balances both.

Deployment. Feature selection shows that the 46 features are redundant: 20 suffice. Combined with the class-weighted Decision Tree, a gateway can run a detector of about 1 MB that classifies a window in about 0.1 µs, leaving the Raspberry Pi's resources for packet capture and feature extraction, which are the real bottlenecks of a live system.

Limitations. (1) A 3% sample was used; the rarest 34-class types have only a few test rows. (2) One dataset from one testbed; transfer to other networks is untested. (3) CICIoT2023 has no replay attacks, which project topic 4 of the course lists. (4) Latency was measured as batch prediction on a Kaggle CPU, not on a Raspberry Pi, and excludes feature extraction. (5) LightGBM was used with default settings; tuning it might make it competitive without rebalancing. (6) The live components of Section 3 (capture, MQTT, dashboard) are designed but not deployed.

7. Conclusion and future work

Following the 10-step IoT design methodology, this project designed a Level 5 gateway intrusion detection system and built and evaluated its detection model on CICIoT2023. The reproduction matched the paper's accuracy and confirmed its blind spot on rare attacks, while finding that its precision and recall are reported swapped. Handling the imbalance on the training data improved the Random Forest's macro-F1 from 0.785 to 0.851 and its rare-class recall from 0.398 to as much as 0.626, all statistically significant. Feature selection kept that quality with 20 of 46 features, and a class-weighted Decision Tree offers nearly the same macro-F1 in about 1 MB.

Future work: deploy the selected model on a Raspberry Pi with live capture and measure end-to-end latency; test transfer to CIC IoT-DIAD 2024 [15] and the Dalhousie IoT 2024 dataset [16]; tune LightGBM for imbalance; and try a two-stage detector that first separates the common floods and then classifies the rest with a specialist model.

References

1. E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A. A. Ghorbani, “CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment,” *Sensors*, vol. 23, no. 13, 5941, 2023. doi:10.3390/s23135941
2. N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
3. H. Han, W.-Y. Wang, B.-H. Mao, “Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning,” in *Proc. ICIC*, LNCS 3644, pp. 878–887, 2005.
4. C. Chen, A. Liaw, L. Breiman, “Using Random Forest to Learn Imbalanced Data,” Tech. Rep. 666, Dept. of Statistics, UC Berkeley, 2004.
5. G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems*, 2017.
6. A. Bahga, V. Madisetti, *Internet of Things: A Hands-On Approach*. VPT / Universities Press, 2014.
7. L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
8. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
9. G. Lemaître, F. Nogueira, C. K. Aridas, “Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning,” *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.
10. Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

11. B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
12. C. Xu, "CICIoT2023" (mirror of the CSV release), Kaggle dataset, version 2, 2025. kaggle.com/datasets/congyuanxu/ciciot2023
13. M. A. Khan et al., "A Deep Learning-Based Intrusion Detection System for MQTT Enabled IoT," *Sensors*, vol. 21, no. 21, 7016, 2021. doi:10.3390/s21217016
14. "An efficient intrusion detection system for MQTT-IoT using enhanced chaotic salp swarm algorithm and LightGBM," *International Journal of Information Security*, Springer, 2023.
15. M. Rabbani et al., "Device Identification and Anomaly Detection in IoT Environments," *IEEE Internet of Things Journal*, 2024.
16. J. Adjei, N. Zincir-Heywood, B. Nandy, N. Seddigh, "Dalhousie NIMS Lab IoT 2024 Dataset," IEEE DataPort, 2024. doi:10.21227/shzy-z570
17. B. C. Ross, "Mutual Information between Discrete and Continuous Data Sets," *PLoS ONE*, vol. 9, no. 2, e87357, 2014.
18. I. Guyon, J. Weston, S. Barnhill, V. Vapnik, "Gene Selection for Cancer Classification using Support Vector Machines," *Machine Learning*, vol. 46, pp. 389–422, 2002.

Appendix A: Reproducibility

Code, tests, results and this report: **github.com/Dhruvsandilya0711/IoT_Mini_Project**. On Kaggle, with the CICIoT2023 CSV dataset [12] attached and Internet enabled, one notebook cell clones the repository and runs `python scripts/kaggle_run.py`, which samples the data, runs every experiment (baseline, improvement, extension, duplicates) and packs `results.zip`. Locally, `python -m pytest` runs the whole pipeline on synthetic data in about 20 seconds. `python scripts/report_figures.py` and `python report/build_report.py` regenerate the figures and this report from the results.